

Inference-aware Node Allocation to Reduce HPC Performance Variability

Arshit Narang*, Ruthvik Tunuguntla*, Pratham Sahu*, Preeti Malakar

Department of Computer Science and Engineering
Indian Institute of Technology Kanpur, India

I. INTRODUCTION

Motivation: High Performance Computing (HPC) systems are foundational to scientific discovery. Despite advances in hardware and interconnect technologies, application performance variability remains a significant challenge. Identical workloads often exhibit different runtimes due to I/O contention, network congestion, and shared resource interference. Contemporary job schedulers do not consider the interference between jobs executing on shared switches. This may lead to increased runtimes and increased performance variability [1] in case of jobs with similar communication and I/O patterns. Thus it is necessary for job schedulers to consider job interference and job I/O characteristics during node allocation. The objective of this work is to reduce performance variability in managed clusters and supercomputers. We propose two complementary job-aware scheduling approaches – (1) a mutual information (MI) based job interference-aware allocation and (2) a CNN-driven I/O-aware allocation strategy.

Limitations of the state-of-the-art approaches: The causes of performance variability in HPC systems have been extensively studied [1] demonstrating that architectural factors, system noise, and resource contention interact to create unpredictable performance across workloads. Recent work on interference-aware scheduling has also focused on I/O-prediction methods. SchedP [2] uses neural networks to predict I/O patterns and adjust job start times, and AI4IO [3] introduces PRIONN for I/O contention prediction and CanarIO for runtime mitigation. These methods rely on coarse-grained, pre-execution metrics and do not capture complex interference dynamics between co-scheduled jobs. Furthermore, the standard SLURM backfill algorithm does not account for historical interference data when placing jobs, which can lead to sub-optimal allocations and increased performance variability.

Key insights and contributions: We propose two node allocation methods that mitigate job interference and improve runtime consistency in HPC systems. The first method considers overall job profiles, whereas the second strategy considers the I/O profiles for scheduling. The first method we propose uses MI score analysis to quantify pairwise interference between jobs based on historical SLURM logs. We compute the mutual information between job performance degradation and co-occurrence patterns to identify job pairs that negatively impact each other. These pairs are then allocated to separate network partitions to minimize interference. The second method constructs I/O behavior images from Darshan logs for each job. It extracts deep features using a compact CNN, and computes similarity scores to guide node allocation. This approach reduces contention by grouping jobs with dissimilar I/O characteristics and avoids co-location of jobs

with similar I/O profiles on adjacent nodes. We obtained a 5.32% reduction of predicted total runtime with the MI-based method on real supercomputing workloads, and a 2.11% reduction in runtime using the CNN-based I/O-aware algorithm, as compared to the default SLURM allocation algorithm.

Limitations of the proposed approach: The MI-based approach provides a holistic view of job interference through statistical correlation but does not identify the specific causes of interference, limiting its ability to implement targeted mitigation strategies beyond separation. The CNN-based method incurs overhead for log processing and inference, which may limit real-time deployment. Future work will investigate online model updating to adapt to evolving workloads and incorporating fine-grained resource monitoring to enhance MI score precision.

II. SIMILARITY-AWARE JOB SCHEDULING

We designed two distinct node allocation strategies to reduce performance variability through interference-aware and I/O-aware job placement. The first method considers overall job profiles, whereas the second strategy considers the I/O profiles for scheduling. The first method leverages Mutual Information (MI) analysis to quantify job interference patterns from historical execution logs. We compute pairwise MI scores between jobs based on co-occurrence patterns (simultaneous execution patterns) and execution time optimality (comparison of job's runtime with average runtime of all similar jobs). Jobs are classified into groups based on similar resource requirements, and the MI score captures the statistical dependency between job co-scheduling and performance degradation. This score guides the allocation algorithm to assign interfering jobs to separate network switches, thereby minimizing contention.

Our second job scheduling policy transforms HPC application I/O logs from Darshan, an I/O profiler, into 12×12 grayscale images, where each pixel represents a normalized I/O feature. We used I/O features such as bytes read/written, access sizes, file open/close counts, and metadata operations (POSIX/MPI). A lightweight CNN architecture extracts deep features from these images, generating a 10-dimensional feature vector that captures the job's I/O signature. We use cosine similarity between feature vectors to determine job similarity. The allocation algorithm places jobs with dissimilar I/O patterns on the same switches while avoiding co-location of jobs with similar I/O profiles.

The overall system architecture employs a two-stage evaluation methodology to assess the effectiveness of each scheduling approach as shown in Figure 1. The first stage involves computing similarity scores (either MI or I/O-based) and implementing the respective allocation strategies. The second stage validates the effectiveness of these allocations using a Random Forest regressor that predicts job completion times based on system state parameters including node bitmaps,

*B.Tech. Students, Indian Institute of Technology Kanpur, India

conflict indicators, switch occupancy, and job metadata. This regression model enables quantitative comparison between our modified allocation strategies and the standard SLURM backfill algorithm by predicting runtime improvements under different allocation scenarios. This is because production supercomputers are unavailable to test modified allocation policies. Thus we required a runtime prediction model for evaluation.

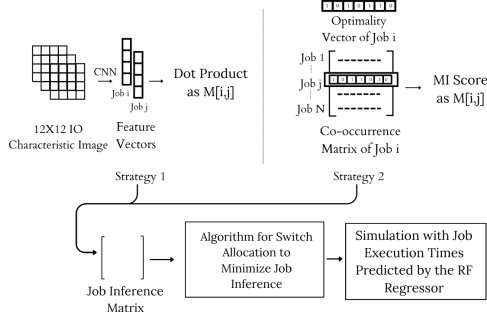


Fig. 1: Workflow of Interference-aware Job Allocation

III. EVALUATION

We evaluated using real production supercomputing traces from two distinct systems to comprehensively assess both proposed allocation strategies. We used the job logs from the PARAM Sanganak supercomputer at IIT Kanpur, which provided detailed historical runtime data and job characteristics necessary for computing MI from 6,000 jobs. We evaluated the I/O-aware CNN-based strategy using Darshan I/O logs and job logs of the Theta supercomputer at Argonne National Laboratory. We used comprehensive I/O behavioral patterns for approximately 10,000 jobs.

The runtimes of some jobs in the job log were close to 0. We implemented a two-stage machine learning pipeline consisting of a binary classifier to distinguish between jobs with near-zero runtimes and jobs with non-zero runtimes followed by regression to predict runtime of the jobs with non-zero runtimes. We implemented the node allocation strategies within the SLURM scheduling framework. The evaluation was done using SLURM simulator. It is not feasible to use a production supercomputer to test our algorithms, thus we predicted the runtimes of jobs using the new algorithms. We used Random Forest regression model to predict job completion times utilizing comprehensive system state features including node allocation bitmaps, conflict indicators representing resource contention, switch occupancy vectors, and job-specific metadata. Figure 2 shows the results of our RF based prediction of runtimes.

Table I shows the accuracy and R^2 score for the two node allocation strategies. Simulation results demonstrated performance improvements for both strategies as compared to standard SLURM FCFS with backfill scheduling. The MI-based allocation strategy reduced total job execution times by 5.32% across $\sim 6,000$ jobs from PARAM Sanganak logs. The I/O-aware CNN approach achieved a 2.11% reduction in total runtime across 10,000 Theta jobs, decreasing from 25,214 seconds to 24,683 seconds. Additionally, the I/O-aware strategy demonstrated substantial improvements in job wait times, reducing average wait time from 6,453 seconds under

default SLURM scheduling to 4,217 seconds with the proposed allocation algorithm.



Fig. 2: Model trained with PARAM Sanganak Logs

Model	Performance
Classifier (Zero vs. Non-Zero Runtime)	Accuracy = 0.7363
Regressor (Non-Zero Runtime Jobs)	R^2 Score = 0.7241
Classifier (Zero vs Non-Zero Runtime)	Accuracy = 0.8937
Regressor (Non-Zero Runtime Jobs)	R^2 Score = 0.7352

TABLE I: Performance of ML models on simulated job data from PARAM Sanganak (top row) and Theta (bottom row) with 85%-15% split.

IV. CONCLUSIONS AND FUTURE WORK

Our work demonstrates that interference-aware node allocation strategies can improve HPC system performance. The MI-based approach achieves 5.32% runtime reduction and the CNN-based I/O-aware method achieves 2.11% runtime reduction. We also observed 35% decrease in job wait times through simulation on PARAM Sanganak and Theta supercomputer workloads. This shows the importance of considering neighbourhood jobs and interference while allocating nodes to jobs. Our future work will focus on online training mechanisms for adaptive allocation strategies, production-grade deployment and evaluation to validate real-world effectiveness. We also plan to incorporate additional system metrics such as memory bandwidth and network topology, extension to heterogeneous computing environments with GPU resources. We plan to development and test lightweight low-overhead implementations suitable for exascale systems.

REFERENCES

- [1] Majid Salimi Beni, Sascha Hunold, and Biagio Cosenza. Analysis and prediction of performance variability in large-scale computing systems. *The Journal of Supercomputing*, 80(10):14978–15005, 2024.
- [2] K. Wu et al. SchedP: I/O-aware Job Scheduling in Large-Scale Production HPC Systems. In *Proceedings of the 19th Annual IFIP International Conference on Network and Parallel Computing*, pages 315–326. Springer, 2022.
- [3] M. R. Wyatt et al. AI4IO: A Suite of AI-Based Tools for I/O-Aware Scheduling. *International Journal of High Performance Computing Applications*, 36(6):618–634, 2022.